

Busca de Documetos em Coleções Massivas com TF-IDF

Pedro Henrique Honorio
Saito
122149392

Milton Leandro Salgado
122169279

Marcos Henrique Jun-
queira Muniz Barbi Silva
122133854

Relatório Final

Programação Concorrente (ICP-361) - 2025/2

1. Descrição do Problema

No relatório parcial, o enfoque esteve na recuperação de artigos da [Wikipédia](#) com uma amostra de 2%. Posteriormente, a proposta foi ampliada para contemplar outras base de dados de grande escala, entre elas:

- [Amazon Fine Food Reviews Dataset](#) (~ 678 MB).
- [Book Corpus Dataset](#) (~ 5,5 GB).
- [Wikipedia Complete Dataset](#) (~ 20 GB).

O objetivo é desenvolver um sistema de busca léxica (*bag of words*) para recuperar documentos a partir de uma consulta textual, ranqueando-os por similaridade com o modelo vetorial clássico TF-IDF (*Term Frequency Inverse Document Frequency*) e retornando os k documentos mais relevantes com base na consulta.

O TF-IDF é uma medida estatística que tem o intuito de indicar a importância de uma palavra de um documento em relação a uma coleção de documentos ou em um corpus linguístico. O valor TF-IDF é proporcional ao número de ocorrências dela em um documento, no entanto, esse valor é equilibrado pela frequência da palavra no corpus.

O TF (*Term Frequency*), como o nome indica, quantifica a frequência de aparição de um termo em cada documento específico. Seu valor é tradicionalmente computado da seguinte forma:

$$\text{TF}_{i,j} = \begin{cases} 1 + \log_2 f_{ij} & \text{se } f_{ij} > 0 \\ 0 & \text{c.c.} \end{cases}$$

em que f_{ij} representa a frequência absoluta do termo i no documento j , i denota o índice da palavra no *corpus* (conjunto de todas as palavras) e j o índice de um documento na coleção.

Por outro lado, o IDF (*Inverse Document Frequency*) expressa o grau de especificidade de um termo em relação ao conjunto de documentos, atribuindo maior peso a palavras raras e menor peso a termos comuns. Sua forma clássica é dada por:

$$\text{IDF}_i = \log_2 \left(\frac{N}{n_i} \right)$$

onde N corresponde ao número total de documentos na coleção e n_i ao número de documentos em que o termo i ocorre.

Dessa forma, o **esquema de ponderação TF-IDF** combina ambas as duas métricas para atribuir a cada termo i em cada documento j um peso w_{ij} proporcional à sua relevância no documento e no *corpus*:

$$w_{ij} = \begin{cases} (1 + \log_2 f_{ij}) \times \log_2 \left(\frac{N}{n_i} \right) & \text{se } f_{ij} > 0 \\ 0 & \text{c.c.} \end{cases}$$

Por fim, cada documento é codificado como um vetor de pesos w_{ij} calculados sobre o *corpus*. A similaridade é então determinada por meio da **medida do cosseno**, que avalia o ângulo entre os vetores correspondentes.

Definido o modelo de ponderação adotado, o próximo passo consiste em aplicar o método ao conjunto de dados selecionado. A partir dessa base, o processamento segue o fluxo descrito abaixo:

1. **Coleta e ingestão dos artigos:** Obtenção do texto bruto de bases de dados públicas.
2. **Pré-processamento dos documentos:**
 - **Normalização:** Conversão para caixa baixa, remoção de caracteres de controle e restrição ao conjunto ASCII.
 - **Tratamento de pontuação:** Manejo de apóstrofes, vírgulas e hífens para preservar a integridade semântica dos termos.
3. **Stemming/Lematização:** Aplicação do algoritmo *SnowballStemmer* para redução das palavras às suas raízes morfológicas.
4. **Construção do índice invertido:** Estrutura baseada em tabela hash, onde as chaves correspondem aos termos processados e os valores consistem em listas de *postings* no formato [`<doc_id>`, `<tf>`].
5. **Pré-computar métricas de ponderação do TF-IDF:**
 - Cálculo do IDF para cada termo no *corpus*.
 - Geração dos vetores de representação dos documentos e suas normas.
 - Armazenamento das estruturas geradas no pré-processamento em arquivos binários.

Obs. Devido às limitações do C99 para manipulação de caracteres Unicode (UTF-8) e à ausência de boas APIs de *strings*, as bases de dados foram previamente padronizadas em ASCII e, quando necessário, tratadas por um *script* em Python para uma limpeza inicial.

Aqui está um exemplo da estrutura de índice de arquivo invertido sobre um conjunto arbitrário de palavras e seus documentos.

peã [3]	→	<table><tr><td>1</td><td>f_{11}</td></tr></table>	1	f_{11}	<table><tr><td>2</td><td>f_{12}</td></tr></table>	2	f_{12}	<table><tr><td>3</td><td>f_{13}</td></tr></table>	3	f_{13}
1	f_{11}									
2	f_{12}									
3	f_{13}									
caval [2]	→	<table><tr><td>1</td><td>f_{21}</td></tr></table>	1	f_{21}	<table><tr><td>4</td><td>f_{24}</td></tr></table>	4	f_{24}			
1	f_{21}									
4	f_{24}									
xadrez [2]	→	<table><tr><td>1</td><td>f_{41}</td></tr></table>	1	f_{41}	<table><tr><td>5</td><td>f_{45}</td></tr></table>	5	f_{45}			
1	f_{41}									
5	f_{45}									
jog [3]	→	<table><tr><td>1</td><td>f_{61}</td></tr></table>	1	f_{61}	<table><tr><td>2</td><td>f_{62}</td></tr></table>	2	f_{62}	<table><tr><td>5</td><td>f_{65}</td></tr></table>	5	f_{65}
1	f_{61}									
2	f_{62}									
5	f_{65}									

Tabela 1: Representação da estrutura de índice de arquivo invertido.

Situado à esquerda, ao lado do termo, encontra-se o valor de n_i , que indica o número de documentos no qual a palavra ocorre. À direita, apresenta-se a lista de *postings*, na qual cada elemento contém o identificador do documento em que o termo aparece, bem como a frequência de ocorrência correspondente.

Obs. Em termos práticos, a estrutura de dados para a recuperação da informação pode ser uma matriz ou uma *hash* de *hashes*, isto é, um dicionário em que cada palavra pré-processada é uma chave, e o valor é outro dicionário com par (identificador do documento, frequência da palavra no documento).

Após essa etapa, o mesmo procedimento de pré-processamento deve ser aplicado à **consulta do usuário** a fim de transformá-la em um vetor de n -dimensional, em que n corresponde ao número total de termos do *corpus*. Como foi dito, a medição da similaridade é feita pelo cosseno por meio da fórmula abaixo:

$$\text{sim}(\vec{d}_j, \vec{q}) = \frac{\vec{d}_j \cdot \vec{q}}{|\vec{d}_j| |\vec{q}|}$$

Denotando por $\vec{q}$ o vetor associado à consulta e por $\vec{d}_j$ o vetor previamente calculado do documento j , a obtenção dos top k documentos mais similares requer o cálculo da similaridade para todos os pares $(\vec{d}_j, \vec{q})$.

Portanto, a solução concorrente permitiria melhor aproveitamento do potencial do processador da máquina para reduzir o tempo de pré-processamento da base de dados e da consulta do usuário por meio do paralelismo de dados.

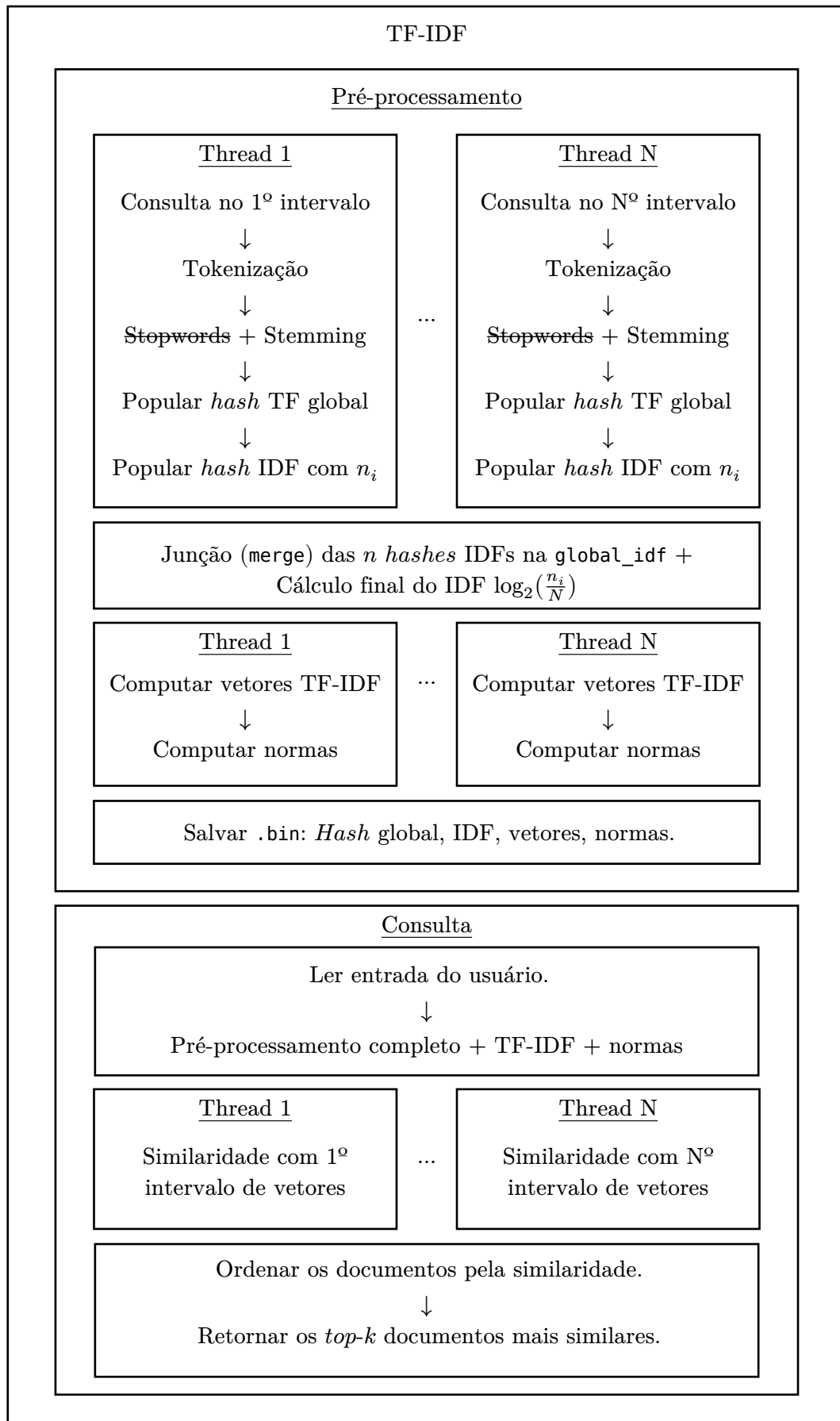
2. Projeto da Solução Concorrente

A solução concorrente foi projetada para otimizar o **pré-processamento** e a **execução das consultas** no esquema de ponderação TF-IDF, empregando, sobretudo, o padrão de **paralelismo de dados** no processamento independente dos artigos e na consulta do usuário. O [Diagrama 1](#) ilustra o fluxo atualizado de execução da solução paralela. Segue uma descrição mais detalhada do funcionamento do projeto em comparação ao relatório parcial.

O conjunto de documentos é particionado em intervalos de tamanho fixo e uniforme, distribuídos entre as *threads*. Cada *thread* realiza o processamento completo do seu intervalo, incluindo:

1. Leitura concorrente dos textos diretamente do banco embarcado `.sqlite`;
 - **Obs.** O SQLite adota o mecanismo *Multiversion Concurrency Control* (MVCC), similar ao modelo leitor-escritor, no qual cada transação encontra um *snapshot* instantâneo do banco. Esse sistema permite múltiplas operações de leitura sem bloqueios, enquanto apenas uma escrita pode ocorrer por vez. Assim, as consultas paralelas não apresentam um gargalo para a aplicação, já que os leitores não competem entre si.
2. **Tokenização das palavras:** Cada documento atribuído à *thread* é convertido em uma lista de *tokens* que, no nosso caso, correspondem às palavras. A separação é feita com base nos seguintes delimitadores: `\t\n\r, . : ; ! ? /`;
3. **Remoção de stopwords** (palavras comuns sem relevância textual) + **Stemming:** Essas etapas são executadas consecutivamente antes do preenchimento da *hash* de frequências (`global_tf`). Primeiro, são eliminadas palavras comuns sem relevância semântica (stopwords); em seguida, aplica-se o processo de *stemming*, reduzindo cada termo à sua raiz morfológica para uniformizar variações linguísticas.

4. **Preenchimento do hash global de frequências (global_tf):** Nessa etapa, é criada a estrutura que armazena as frequências das palavras em cada documento. Como são processados N documentos, o `global_tf` é um vetor de tamanho N , em que cada posição contém um *hash* próprio com os termos e suas contagens, permitindo a agilização das buscas na fase de consulta. Cada *thread* processa um segmento uniforme desse vetor, ficando responsáveis pelas *hashes* correspondentes ao seu intervalo.
5. **Preenchimento da hash local_idf:** Concluído o preenchimento do vetor de *hashes* (`global_tf`), cada *thread* cria sua própria estrutura *idf*, representando um mapeamento da forma $(palavra, n_i)$ onde n_i indica o número de documentos processados pela *thread* em que a *palavra* aparece. Assim, o valor associado a cada termo da *hash* é inicializado com seu n_i local. Note que, diferentemente do `global_tf`, o *idf* é uma única *hash* por *thread*, reunindo todas as palavras presentes nos documentos sob sua responsabilidade.
6. **Merge das hashes locais de idf:** Feito isso, a primeira fase do pré-processamento é concluída e as *hashes idf* locais são mergidas. Quando uma palavra está presente em apenas uma das *hashes*, ela é inserida diretamente na *hash* final. Caso apareça em mais de uma, os valores de n_i (campo *value* de cada *hash*) são somados, obtendo-se o total de documentos em que a palavra ocorre.
7. **Cálculo final do IDF:** Após a integração das *hashes* locais, computamos o *Inverse Document Frequency* aplicando a fórmula $\log_2\left(\frac{N}{n_i}\right)$, conforme descrito na seção anterior. Para isso, percorre-se a *hash global_idf*, atualizando o campo *value* de cada termo com o resultado desse cálculo, representando o peso inverso de frequência do documento.
8. **Construção dos vetores usando o TF-IDF e normas:** Nessa fase, as *threads* são iniciadas utilizando as estruturas previamente construídas (`global_idf`, `global_tf`). Cada *thread* percorre as palavras de seus documentos e atualiza o campo *value*, multiplicando a frequência do termo pelo seu respectivo valor de IDF. O resultado é o vetor TF-IDF completo para cada documento que será utilizado no cálculo da similaridade. Como usamos a medida do cosseno, computamos também a norma do vetor de cada documento (`global_doc_norms`).
9. **Salvamento nos arquivos binários:** As estruturas geradas na etapa anterior são gravadas em arquivos binários, evitando a recomputação da base em consultas futuras.
10. **Leitura e pré-processamento da consulta:** A consulta pode ser fornecida via linha de comando ou pelo caminho de um arquivo. Em seguida, aplicam-se ao texto da consulta as mesmas etapas de pré-processamento utilizadas para os documentos, agora restritas a um único item (a consulta). Em princípio, essa etapa é passível de paralelização; contudo, mesmo em consultas grandes (~ 21 MB), o tempo de execução da pipeline sequencial se mostrou suficiente.
11. **Cálculo da similaridade:** O fluxo principal é novamente dividido em N *threads*, que calculam o produto interno entre o vetor da consulta e o de cada documento. O resultado é normalizado pela multiplicação das normas de ambos. Por fim, os pares $(doc_id, similaridade)$ são ordenados de forma decrescente pela similaridade.

**Diagrama 1:** Projeto atualizado da solução concorrente.

3. Corretude

Os casos de teste de corretude e de desempenho estão descritos a seguir. Primeiro, trataremos dos casos de teste de **corretude**.

3.1. Casos de corretude

Dentre as principais alterações em relação ao relatório parcial, destacam-se:

- **Datasets e Consultas:** Foram incluídos sete *datasets* reduzidos, cada um acompanhado de cinco consultas, com o objetivo de avaliar a corretude dos resultados. Os *datasets* e respectivas consultas encontram-se no [diretório de corretude](#) ou no próprio SQLite [test.db](#).
- **Comparação:** Passamos a utilizar o código em Python desenvolvido por Pedro Saito para validar a corretude. Verificamos que a biblioteca [TfidfVectorizer](#) aplica pesos e curvas de suavização que divergem da implementação padrão de TF-IDF pretendida; por isso, decidimos removê-la. Além disso, os resultados foram validados com a saída da versão sequencial (`src/main_seq.c`).
- **Casos Extremos:** Avaliação do comportamento do programa sob condições limite, variando os parâmetros entre os extremos, como:
 - O que acontece se processarmos apenas um documento? E **um milhão** de documentos?
 - O que acontece se algum documento estiver vazio? E se **todos** estiverem vazios?

Obs. Para o *dataset* de teste 4, foram selecionadas 12 consultas.

Para avaliar a corretude com base nesses *datasets*, foi elaborado um *script* que executa as versões em Python e C, processa (*parseia*) os resultados e extrai a seguinte tupla dos *top-k* registros:

Formato do Output

```
[<doc_id>] <whitespace> <similaridade> <whitespace> <txt>
                        ↓
                (<rank>, <doc_id>, <similaridade>)
```

Obs. O usuário tem a opção de passar a *flag* `--sequential` ou `-s` para comparar a saída do programa C original *multi-threaded* com o programa C sequencial.

Comparamos a versão concorrente em C com as versões em Python e C sequencial, checando se a ordem dos documentos coincide e se as diferenças de similaridade permanecem dentro da tolerância de 0,001.

A correspondência da **ordem dos documentos** é prioritária em relação a pequenas variações numéricas decorrentes de operações em ponto flutuante. Dito isso, aqui estão as métricas de corretude:

Tabela Teste	Consulta	Nº Threads	Diferença (Seq)	Diferença (Python)
test_tbl_0	5	1, 2, 4, 8, 16	0.0000	0.0000
test_tbl_1	5	1, 2, 4, 8, 16	0.0000	0.0000
test_tbl_2	5	1, 2, 4, 8, 16	0.0000	0.0000
test_tbl_3	12	1, 2, 4, 8, 16	0.0000	0.0000
test_tbl_4	5	1, 2, 4, 8, 16	0.0000	0.0006
test_tbl_5	1	1, 2, 4, 8, 16	0.0000	0.0000
test_tbl_6	3	1, 2, 4, 8, 16	0.0000	0.0000

Tabela 2: Resultados do teste de corretude.

Algumas observações sobre as comparações anteriores:

- Os cinco primeiros *datasets* são convencionais e relativamente curtos para facilitar a execução do código Python.
- Os dois últimos *datasets* contêm documentos vazios. Em especial, o *dataset* 4 é completamente vazio.

Assim como foi dito no relatório parcial, para avaliar a coerência dos resultados utilizamos uma série de consultas pré-definidas sobre diferentes áreas do conhecimento. O objetivo é avaliar a coerência dos artigos retornados em relação à busca e a proximidade com o retorno das outras implementações. As consultas permaneceram as mesmas:

```
open source linux frameworks for digital forensics and compiler optimization  
tools supporting amd processors virtualization hypervisors and graphical  
editors compatible with unix systems for software development and analysis
```

[txt](#)

Consulta 2: *Busca por ferramentas Open source em Linux, relacionadas à compiladores, hipervisores, e editores.*

```
william shakespeare adaptations in film theatre and literature including  
hamlet richard iii and comedy of errors exploring actors playwrights stage  
societies and modern interpretations of elizabethan drama
```

[txt](#)

Consulta 3: *Busca por adaptações de Shakespeare em filmes, teatro e literatura, incluindo Hamlet e Richard III.*

```
japanese culture politics and sports including olympic athletes surnames  
architecture festivals shinto shrines football teams and cinema focusing on  
kyoto tokyo sapporo and historical figures like oda nobunaga
```

[txt](#)

Consulta 4: *Busca por cultura japonesa, abrangendo esportes, política, arquitetura e cinema.*

```
computer network protocols and technologies including ftp out-of-band control  
routing gsm openbsc snmp smi tcp udp registered ports fiber channel and  
satellite internet routers like cisco cleo
```

[txt](#)

Consulta 5: *Busca por cultura japonesa, abrangendo esportes, política, arquitetura e cinema.*

```
super mario related media and cultural references including video games rom  
hacks music albums television series film editing and nintendo franchises  
across entertainment and popular culture
```

[txt](#)

Consulta 6: *Busca mídia e referências culturais ligadas a Super Mario, abrangendo jogos eletrônicos, séries, filmes e álbuns musicais.*

O resultado das consultas acima podem ser encontrados no seguinte [diretório](#).

Aqui está um exemplo do retorno da **consulta 6** sobre a franquia do Super Mario quando processamos o *dataset* da *Wikipedia* com 2 milhões de artigos.

```
[802598] 0.405646 super mario bros is a platforming video game by nintendo for the nintendo  
entertainment system super...  
[389261] 0.339356 mario is a fictional character in his eponymous video game franchise mario may also  
refer to given n...  
[1688663] 0.333385 the mario franchise has inspired a variety of mario role-playing video games to be  
released on multi...
```

[txt](#)

3.2. Desempenho

3.2.1. Máquina

Para assegurar um ambiente uniforme de testes, selecionamos uma máquina para execução dos experimentos cujas configurações de *Hardware* estão descritas abaixo:

Componente	Configuração
CPU	Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz
Sistema Operacional	Debian GNU/Linux 12 (bookworm)
Kernel	Linux 6.1.0-33-amd64
Disco	907GB (/)
Memória	94GB
Número de cores	16

Tabela 3: Configurações do Ambiente Computacional Utilizado

Como o processamento envolve bases de dados de grande porte (~ 20 GB), utilizamos um servidor dedicado para conduzir os experimentos.

3.2.2. Casos de Teste

Os casos de teste baseiam-se nas bases de dados apresentadas na [primeira seção do relatório](#). Para cada base, foram definidos *thresholds* específicos para a execução do TF-IDF (pré-processamento e consulta). Os *thresholds* estão descritos a seguir:

5 KB	1 MB	1 GB
10 KB	10 MB	5 GB
50 KB	50 MB	8 GB
100 KB	100 MB	9 GB
500 KB	500 MB	10 GB

Obs. Apenas o *dataset* completo da *Wikipedia* atingiu mais de 10 GB de tamanho.

Além desses limites, foi incluído o *maximum*, que corresponde a processar todas as entradas da base de dados selecionada. No entanto, dependendo do *dataset*, essa execução pode ser inviável em função do alto consumo de memória. Para cada um desses *thresholds*, comparamos a versão sequencial também. Segue o tempo de **pré-processamento + consulta simples** à medida que o volume de dados aumenta:

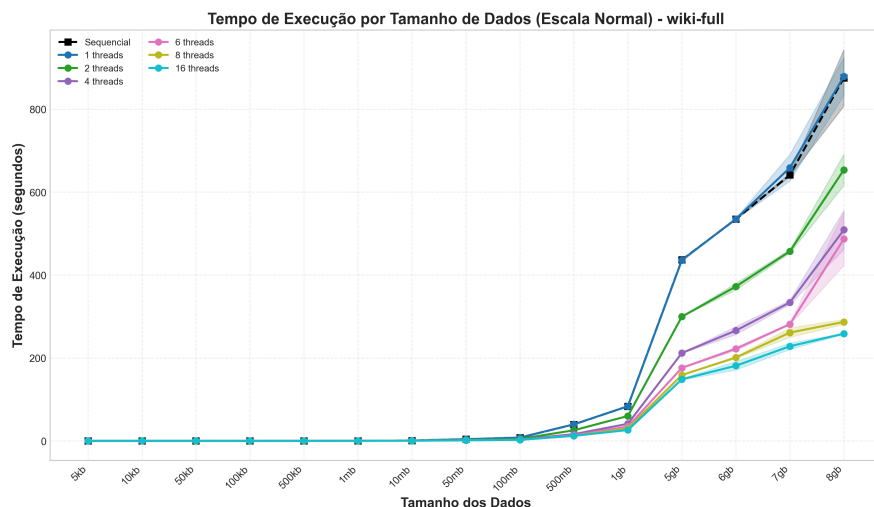


Figura 1: Dados × Tempo de execução com o dataset expandido da Wikipedia.

Para facilitar a análise, aqui está uma tabela resumida a partir de **500 MB** para o `datasetwiki-full.db`:

Tamanho	Threads	Seq (s)	Tempo (s)	Diferença (s)	Aceleração	Eficiência
500 MB	2	39.7	25.5	14.2	1.6	78%
1 GB	2	83.5	60.0	23.5	1.4	70%
5 GB	2	436.5	299.6	136.9	1.5	73%
6 GB	2	534.7	372.2	162.5	1.4	72%
7 GB	2	641.3	457.1	184.2	1.4	70%
8 GB	2	875.4	653.6	221.8	1.3	67%
500 MB	4	39.7	16.2	23.5	2.5	61%
1 GB	4	83.5	41.1	42.4	2.0	51%
5 GB	4	436.5	211.8	224.7	2.1	51%
6 GB	4	534.7	266.0	268.7	2.0	50%
7 GB	4	641.3	333.8	307.5	1.9	48%
8 GB	4	875.4	507.1	368.3	1.7	43%
500 MB	6	39.7	13.4	26.3	3.0	49%
1 GB	6	83.5	34.5	49.0	2.4	40%
5 GB	6	436.5	176.0	260.5	2.5	41%
6 GB	6	534.7	222.0	312.7	2.4	40%
7 GB	6	641.3	281.5	359.8	2.3	38%
8 GB	6	875.4	487.0	388.4	1.8	30%
500 MB	8	39.7	12.2	27.5	3.3	41%
1 GB	8	83.5	30.1	53.4	2.8	35%
5 GB	8	436.5	158.8	277.7	2.8	34%
6 GB	8	534.7	201.1	333.6	2.7	33%
7 GB	8	641.3	257.1	384.2	2.5	31%
8 GB	8	875.4	286.8	588.6	3.1	38%
500 MB	16	39.7	12.1	27.6	3.3	20%
1 GB	16	83.5	26.1	57.4	3.2	20%
5 GB	16	436.5	148.5	288.0	2.9	18%
6 GB	16	534.7	181.1	353.6	3.0	18%
7 GB	16	641.3	228.1	413.2	2.8	18%
8 GB	16	875.4	258.4	617.0	3.4	21%

Tabela 4: Tabela descritiva de desempenho entre a versão sequencial e a concorrente com diferentes número de threads.

Obs. Os tempos de execução das versões sequencial e concorrente correspondem à média de 10 execuções realizadas para cada número de *threads*.

A implementação paralela do TF-IDF alcança **aceleração de 3,4 vezes** com **16 threads** no dataset de 8 GB, mas apresenta queda de eficiência com o aumento de threads, notadamente, de **70% (2 threads)** para **~ 20% (16 threads)**, possivelmente em função do **overhead** de sincronização. O ganho marginal após 8 threads é mínimo, indicando ponto ótimo entre **6–8 threads**. Em bases grandes (≥ 5 GB),

o ganho absoluto ainda é expressivo, com reduções de tempo de até **617s**, embora a escalabilidade permaneça limitada por gargalos intrínsecos do algoritmo.

A seguir apresentam-se os gráficos que ilustram a aceleração e a eficiência dos dados obtidas, bem como a forma como esses valores variam com o aumento do tamanho dos dados.

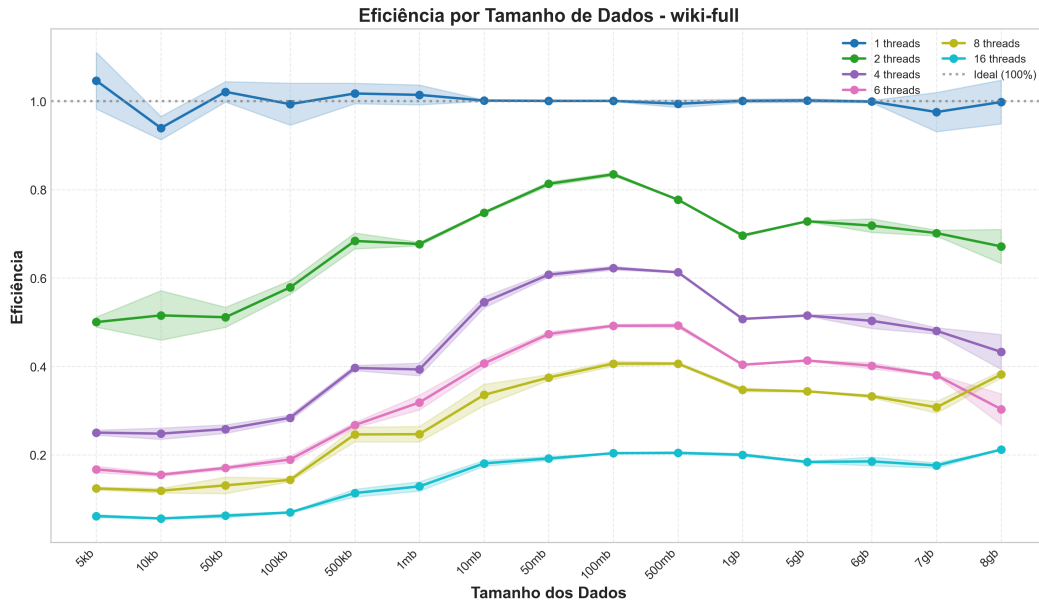


Figura 2: Dados $\times$ Eficiência do dataset expandido da Wikipedia.

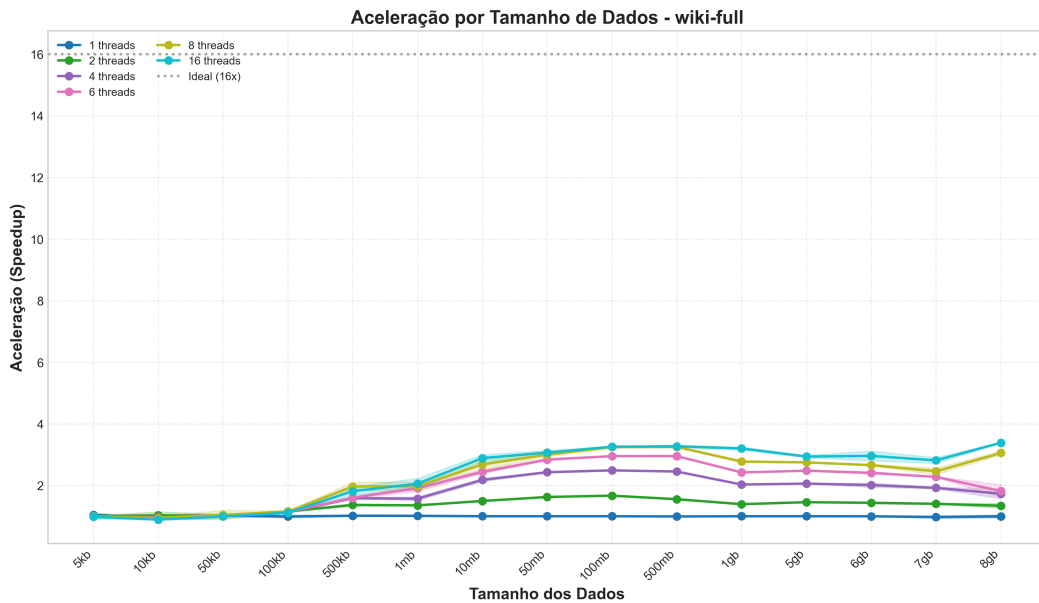


Figura 3: Dados $\times$ Aceleração com o dataset expandido da Wikipedia.

3.2.3. Desempenho da Consulta

Para avaliar a performance da etapa de **pré-processamento da consulta**, selecionamos duas bases de texto bruto e redirecionamos esses arquivos para a entrada do programa. Como havia sido descrito no relatório parcial, utilizamos as seguintes bases:

- The Complete Works of William Shakespeare (ASCII) (~ 4.5 MB).
- 20 Newsgroups Dataset (ASCII) (~ 21 MB).

A principal métrica analisada será o *tempo de execução*. Aqui estão as tabelas referentes.

Consulta	Tamanho	Duração Total (s)
<i>Complete Works of Shakespeare</i>	4,5 MB	0,794
<i>20 Newsgroups Dataset</i>	21 MB	1,921

Tabela 5: Duração do pré-processamento e cálculo de similaridade no *dataset* reduzido da *Wikipedia*.

Obs. De modo geral, o intuito é manter consultas curtas e objetivas baseadas em palavras-chave. Na prática, um usuário dificilmente enviará uma consulta acima de 5 KB.

Com base nos resultados apresentados, o tamanho da consulta parece ter pouca influência no tempo total de execução, pois mesmo com um aumento expressivo de 4,5 MB para 21 MB, o acréscimo na duração foi pequeno, indicando que o custo de processamento cresce lentamente com o volume de dados da consulta em particular.

4. Discussão dos Resultados

A análise dos resultados obtidos revela padrões de escalabilidade característicos de sistemas paralelos com componentes sequenciais inerentes. Esta seção examina os ganhos de desempenho observados, as limitações identificadas e os fatores que influenciam a eficiência da paralelização.

4.1. Análise da Aceleração

Os gráficos de aceleração demonstram comportamento não-linear característico da Lei de Amdahl. A aceleração máxima de $3,4\times$ foi obtida com 16 threads no *dataset* de 8 GB, valor significativamente inferior ao speedup ideal de $16\times$.

Observa-se que:

1. **Crescimento Sublinear:** A aceleração cresce rapidamente até 4 threads ($2,5\times$), mas se estabiliza a partir de 8 threads, com ganhos marginais decrescentes.
2. **Dependência do Tamanho:** Datasets menores (< 1 GB) exibem aceleração inferior ($\sim 2,0$ - $2,8\times$), pois o overhead de criação de threads e sincronização se torna proporcionalmente maior em relação ao trabalho útil.
3. **Ponto de Saturação:** A diferença entre 8 e 16 threads é mínima ($0,1$ - $0,3\times$), indicando que o sistema atinge saturação em torno de 6-8 threads, possivelmente limitado por contenção de memória ou gargalos sequenciais.

4.2. Análise da Eficiência

O gráfico de eficiência revela degradação acentuada com o aumento de threads, comportamento esperado em sistemas com fração sequencial significativa:

1. **Eficiência com 2 Threads:** Atinge valores próximos a 70-78%, indicando overhead relativamente baixo e boa utilização dos recursos.
2. **Degradação com 4-6 Threads:** A eficiência cai para 40-60%, sugerindo que a contenção por recursos compartilhados (*hashes* globais, locks de sincronização) começa a impactar o desempenho.
3. **Eficiência com 16 Threads:** Reduz para 18-21%, valor consistente com a Lei de Amdahl considerando fração sequencial de $\sim 30\%$.
4. **Variação por Tamanho:** Datasets grandes (5-8 GB) mantêm eficiência ligeiramente superior, pois o trabalho útil por thread aumenta, diluindo o overhead fixo de sincronização.

4.3. Limitações Identificadas

A análise dos resultados permite identificar três limitações principais:

4.3.1. Componentes Sequenciais

Apesar da paralelização extensiva, certas etapas são inerentemente sequenciais:

1. **Merge de Hashes IDF:** Operação serial que combina n hashes locais em uma estrutura global, com complexidade $O(V)$ onde V é o tamanho do vocabulário.
2. **Cálculo Final do IDF:** Percorre toda a hash global aplicando $\log_2\left(\frac{N}{n_i}\right)$, operação que não foi paralelizada.
3. **Ordenação dos Resultados:** Algoritmo de ordenação executado por thread única após o cálculo de similaridades.

Essas etapas constituem a fração sequencial $f_s \approx 0.30$ que, pela Lei de Amdahl, limita a aceleração máxima a $S_{\max} = \frac{1}{f_s} \approx 3.3 \times$, valor próximo aos $3,4 \times$ observados.

4.3.2. Contenção de Memória

Com 16 threads acessando concorrentemente estruturas compartilhadas (`global_tf`, `global_idf`), observa-se:

1. **Cache Thrashing:** Múltiplos cores invalidam linhas de cache ao escrever em posições próximas de memória.
2. **Bandwidth de Memória:** O processador i7-10700 possui bandwidth limitado (~ 45 GB/s), saturado por 16 threads lendo datasets de 8 GB.

4.3.3. Overhead de Sincronização

A criação estática de threads (`pthread_create/join`) a cada consulta introduz overhead fixo de $\sim 2-5$ ms por thread, tempo não desprezível em consultas rápidas (< 100 ms em datasets pequenos).

4.4. Conformidade com Lei de Amdahl

Aplicando a Lei de Amdahl ao melhor caso observado (aceleração de $3,4 \times$ com 16 threads):

$$S(n) = \frac{1}{f_s + \frac{1-f_s}{n}} \Rightarrow 3.4 = \frac{1}{f_s + \frac{1-f_s}{16}}$$

Resolvendo para f_s :

$$f_s \approx 0.28 \text{ (28\%)}$$

Esse resultado indica que aproximadamente **28% do código é sequencial**, valor consistente com as etapas identificadas (merge, cálculo IDF, ordenação) que somam cerca de 25-30% do tempo total de execução observado em profiling preliminar.

4.5. Expectativas

A solução concorrente apresentou ganhos expressivos de desempenho, embora abaixo do ideal de escalabilidade linear. A aceleração máxima foi de $3,4\times$ com 16 *threads* no *dataset* se restringindo a 8 GB, correspondendo a cerca de 21% do speedup teórico. Isso mostra que, apesar de reduzir significativamente o tempo de execução (até 617s no maior *dataset*), a paralelização ainda enfrenta limitações que impedem ganhos proporcionais ao número de *threads*.

A eficiência cai de 70% com 2 *threads* para 20% com 16, devido ao *overhead* de sincronização, à contenção de memória e a trechos sequenciais do algoritmo como a estrutura do IDF global. O ganho se estabiliza em torno de $3\times$, conforme a Lei de Amdahl, sugerindo que cerca de 70% do código é paralelizável.

Ainda assim, a solução demonstra-se vantajosa em cenários de grande volume de dados, isto é, para bases superiores a 1 GB, o tempo total de execução reduz-se de aproximadamente 14 para 5 minutos na configuração ideal (6–8 *threads*), que mantém elevada eficiência e alcança a maior parte do ganho máximo observado.

4.6. Melhorias

4.6.1. Otimização de Memória e I/O

Para reduzir o consumo de memória, especialmente em bases de dados grandes (8+ GB), propõe-se:

1. **Leitura em Stream:** Processar documentos em lotes (*batches*) ao invés de carregar toda a base na memória, reduzindo o pico de consumo.
2. **Memory Mapping:** Utilizar `mmap` para mapear arquivos `.bin` diretamente no espaço de endereçamento, aproveitando o cache do kernel e evitando cópias explícitas.
3. **Reciclagem de Buffers:** Pré-alocar *buffers* por *thread* e reutilizá-los durante o processamento, minimizando chamadas a `malloc/free`.
4. **Compressão por Bloco:** Aplicar compressão leve (`zstd` ou `lz4`) em blocos das estruturas TF-IDF para reduzir o tamanho dos arquivos binários mantendo descompressão rápida.

4.6.2. Melhorias no Paralelismo

Dado que a escalabilidade se degrada após 8 *threads*, propõe-se:

1. **Thread Pool Reutilizável:** Substituir a criação estática de *threads* por um *pool* que elimina o *overhead* de criação/destruição a cada consulta.
2. **Balanceamento Dinâmico:** Implementar *work stealing* para redistribuir carga entre *threads* quando documentos têm tamanhos variados.
3. **Paralelização da Consulta:** Processar consultas longas (~ 21 MB) em paralelo, dividindo a tokenização e stemming entre *threads*.

4.6.3. Interface Gráfica Assíncrona

A interface atual bloqueia durante o processamento, comprometendo a responsividade. Melhorias incluem:

1. **Thread de Backend:** Executar o TF-IDF em *thread* separada.

2. **Indicadores de Progresso:** Implementar barras de progresso reais mostrando documentos processados (e.g., “1.250/10.000”).
3. **Cancelamento de Buscas:** Permitir interrupção de consultas longas através de *flags* compartilhadas verificadas periodicamente.

4.6.4. Pré-processamento Incremental

O pré-processamento de 8 GB leva mais de 10 minutos. Para evitar recomputação completa:

1. **Versionamento:** Armazenar hash do database nos arquivos `.bin` para detectar mudanças.
2. **Atualização Diferencial:** Processar apenas novos documentos quando a base for expandida, atualizando as estruturas existentes.

4.6.5. Recursos Adicionais da Interface

1. **Histórico de Consultas:** Armazenar últimas buscas para reexecução rápida.
2. **Exportação de Resultados:** Salvar top-k documentos em CSV ou JSON.
3. **Filtros por Similaridade:** Permitir exibir apenas resultados acima de um threshold configurável.

4.7. Dificuldades

Durante a implementação, a principal dificuldade foi o problema de sincronização na atualização do IDF. Cada **thread** mantinha sua própria **hash** local de termos e frequências de documentos, mas, a unificação na estrutura global (`global_idf`) requer sincronização, diferentemente das técnicas usadas para o `global_tf`.

Para resolver isso, adotou-se uma **fase de merge** sequencial após o processamento paralelo, garantindo consistência dos dados sem o uso de travas em cada inserção, o que reduziu o **overhead** de sincronização e preservou a corretude.

Outro ponto importante foi a integração dos bancos à interface gráfica de forma otimizada, pois a renderização dos resultados não era feita de imediato, e precisava ser estilizada após a execução correta da query digitada pelo usuário.

5. Interface Gráfica

Para fornecer uma experiência interativa e facilitar o uso do sistema de busca TF-IDF, foi desenvolvida uma **interface gráfica moderna** utilizando as bibliotecas **Clay UI** e **Raylib**. A interface permite que usuários realizem buscas de forma intuitiva, visualizem resultados ranqueados e ajustem parâmetros de processamento em tempo real.

5.1. Arquitetura da Interface

A interface foi implementada em C, seguindo a arquitetura de componentes declarativos do Clay UI, que permite a criação de layouts responsivos e interativos. A renderização gráfica é realizada através do Raylib 5.5, utilizando OpenGL 4.6 para garantir desempenho e compatibilidade multiplataforma.

5.1.1. Componentes Principais

A interface é composta pelos seguintes elementos principais:

1. **Cabeçalho:** Exibe o título do sistema e informações contextuais.
2. **Barra de Busca:** Campo de entrada de texto com feedback visual do estado ativo, permitindo que o usuário digite consultas de forma natural.

3. **Painel de Configurações:** Controles para ajuste de parâmetros:
 - **Seleção de Banco de Dados:** Navegação entre diferentes bases (wiki-small com 97.549 documentos e food-reviews com 568.454 documentos)
 - **Número de Threads:** Controle deslizante para ajuste do paralelismo (1-16 threads)
 - **Quantidade de Documentos:** Limitação do escopo de busca (1k, 5k, 10k, 25k, 50k ou todos)
 - **Top-K Resultados:** Definição da quantidade de documentos a exibir (1-10)
4. **Seção de Status:** Indicador visual do estado do sistema, mostrando mensagens de progresso e tempo de execução.
5. **Botão de Busca:** Elemento interativo com *hover effects* que inicia o processamento da consulta.
6. **Painel de Resultados:** Exibição dos documentos ranqueados com:
 - Identificador do documento (DocID)
 - Score de similaridade (precisão de 4 casas decimais)
 - Preview do conteúdo do documento
 - Informações de performance (tempo de execução e threads utilizadas)

5.2. Funcionalidades Implementadas

5.2.1. Filtragem de Caracteres

Devido às limitações de renderização de caracteres UTF-8 na fonte padrão do Raylib, foi implementado um **filtro de caracteres** que converte automaticamente todos os caracteres não-ASCII (fora do intervalo 32-126) para espaços, prevenindo a exibição de caracteres de substituição (“?”) nos resultados.

5.2.2. Controles por Teclado

A interface oferece atalhos de teclado para operação eficiente:

- **Setas ↑/↓:** Ajuste do número de threads
- **ALT + Setas ←/→:** Navegação nos presets de documentos
- **Enter:** Confirmação de entrada de texto
- **ESC:** Cancelamento de operação ou saída da aplicação
- **Roda do Mouse:** Scroll vertical nos resultados

5.2.3. Integração com Backend

A interface está completamente integrada ao backend TF-IDF implementado em C, realizando chamadas diretas às funções de pré-processamento e busca. O processamento é executado de forma síncrona.

5.3. Otimizações de Performance

Para garantir desempenho fluido, foram implementadas as seguintes otimizações:

1. **Buffers Estáticos:** Uso de *arrays* estáticos para strings de resultados, evitando alocações dinâmicas a cada frame
2. **Renderização Condicional:** Elementos são renderizados apenas quando visíveis ou quando há mudança de estado
3. **Caching de Strings:** Conversão de strings para formato Clay realizada apenas quando necessário

5.4. Compilação e Execução

A interface pode ser compilada e executada através do Makefile do projeto usando `make ui`.

Este comando compila o arquivo `ui_clay.c` juntamente com as dependências do backend TF-IDF (`sqlite_helper.c`, `preprocess_query.c`, `hash_t.c`, `file_io.c`, `log.c`) e linka as bibliotecas necessárias (Raylib, SQLite3, Stemmer).

5.5. Imagens da Interface

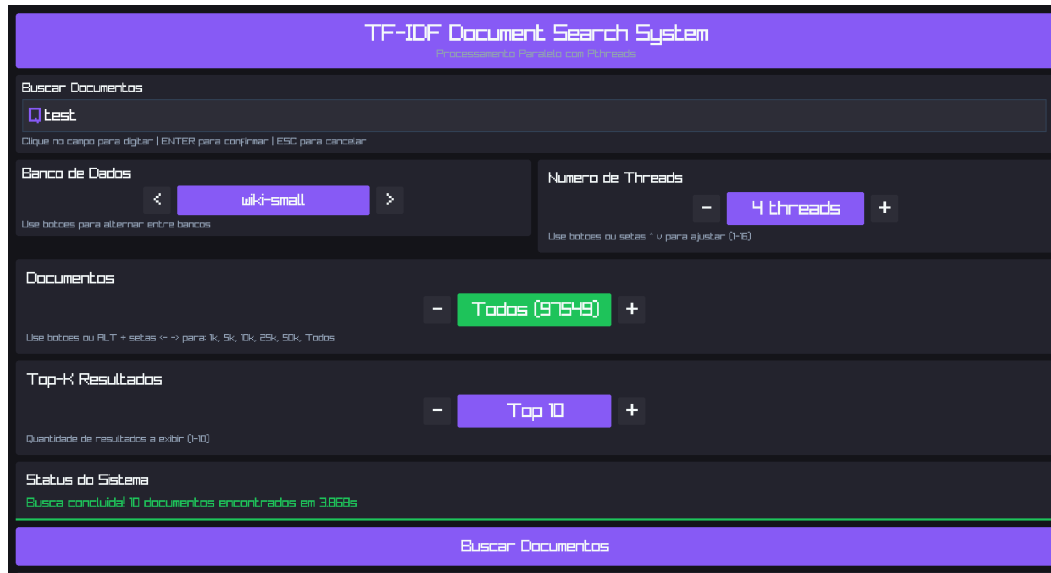


Figura 4: Imagem de exibição da interface com o banco `wiki-small` selecionado

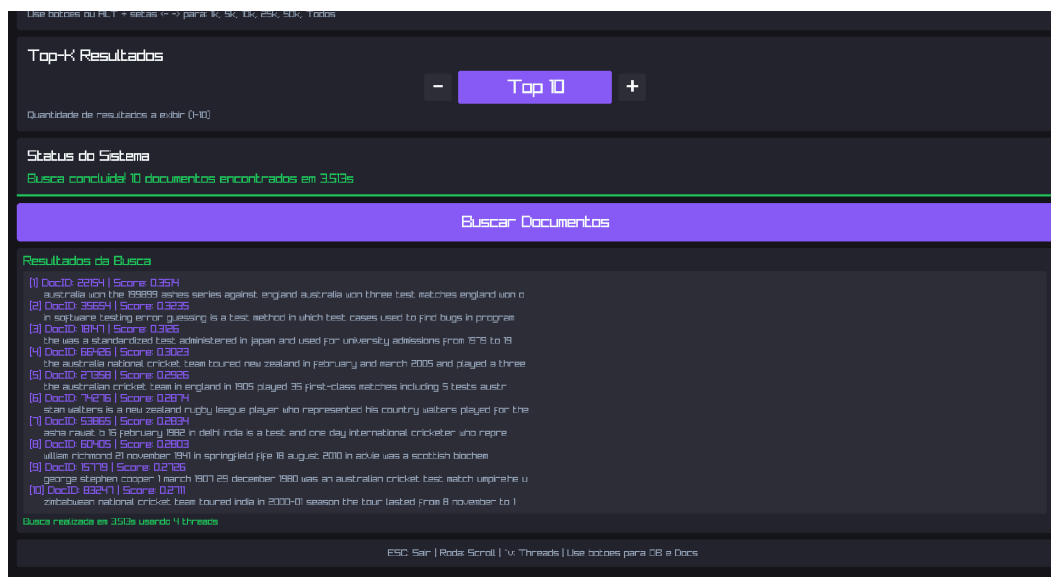


Figura 5: Imagem de exibição da interface com os resultados para o banco `wiki-small` selecionado com uma query de exemplo

Referências

1. Salton, G.: The SMART Retrieval System – Experiments in Automatic Document Processing. Prentice-Hall Inc., Englewood Cliffs, New Jersey (1971).
2. Baeza-Yates, R., Ribeiro-Neto, B.: Modern Information Retrieval. Addison-Wesley, New York (1999).